Screenshots — Terraform Modules

Lecture 9 · 35 shots, chronological · source stills from the session (2026-10-06 21:47 – 23:09)

Terraform Modules

A *Terraform module* is a collection of standard configuration files in a dedicated directory. **Terraform modules are the main feature that allows us to reuse resource definitions across multiple projects or simply have a better organization in a single project**. This is much like what we do in standard programming: instead of a single file containing all code, we organize our code across multiple files and packages.

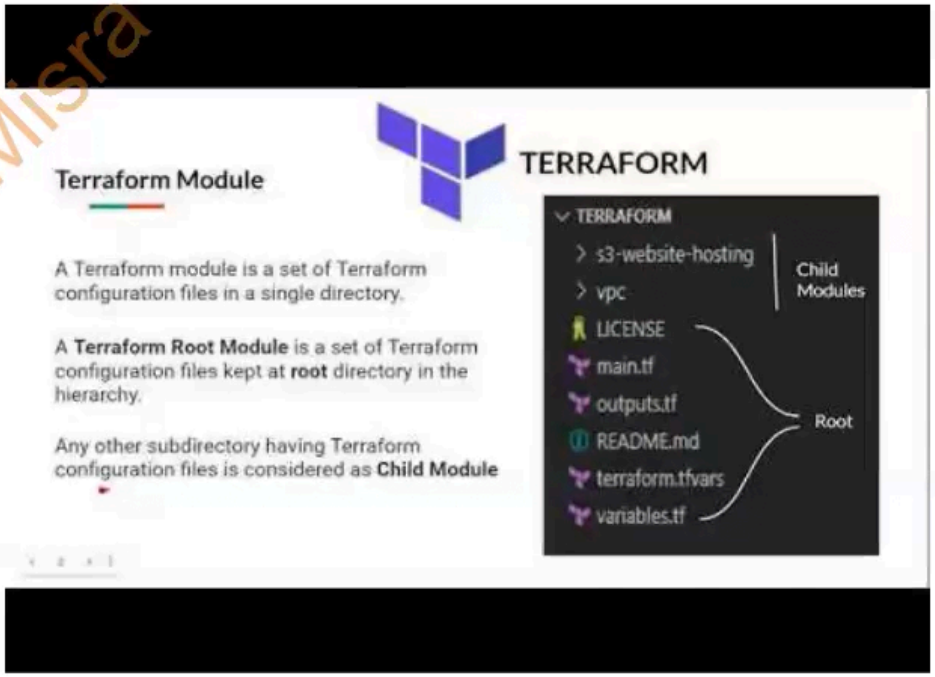
So in simple term "modules allows you to group resources togather and reuse this group latter, possibly many times"

Terminology

1. **Module**:
   A Terraform module is a set of Terraform configuration files in a single directory.

   Even a simple configuration consisting of a single directory with one or more .tf files is a module. So in this sense, every Terraform configuration is part of a module.

2. **root module**:
   When we run Terraform commands directly from a directory, it is considered the **root module**.

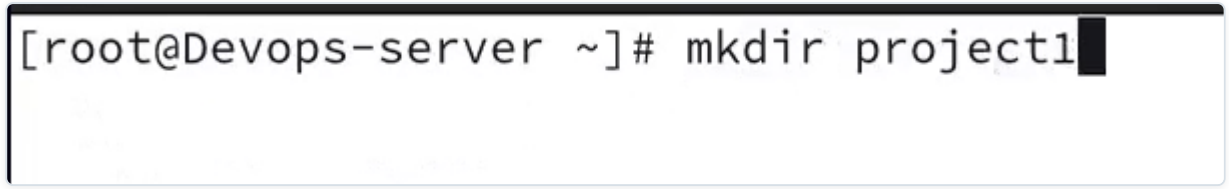


Ram Misra

A **Terraform module** is a self-contained collection of multiple Terraform configuration files stored together in a single directory. It acts as a **reusable building block** that abstracts and groups related infrastructure resources together into a single logical unit, working similarly to functions in programming languages. HashiCorp Developer +4

The syntax for using a module is:

```
module "<NAME>" {
  source = "<SOURCE>"
  [CONFIG ...]
}
```


[root@Devops-server ~]# mkdir project1


[root@Devops-server project1]# mkdir modules

```
[root@Devops-server modules]# mkdir ec2_modules vpc_modules
```

```
[root@Devops-server ~]# mkdir project1
[root@Devops-server ~]# cd project1/
[root@Devops-server project1]# mkdir modules
[root@Devops-server project1]# cd modules/
[root@Devops-server modules]# mkdir ec2_modules vpc_modules
```

root@Devops-server:~/project1 — vim s3.tf

```
resource "aws_s3_bucket" "first" {
  bucket = "ramprateek"
  tags = {
        Name = "my_buck"
}
}
```

Child module `ec2_modules/varibale.tf` — `instance_count` number, default 1

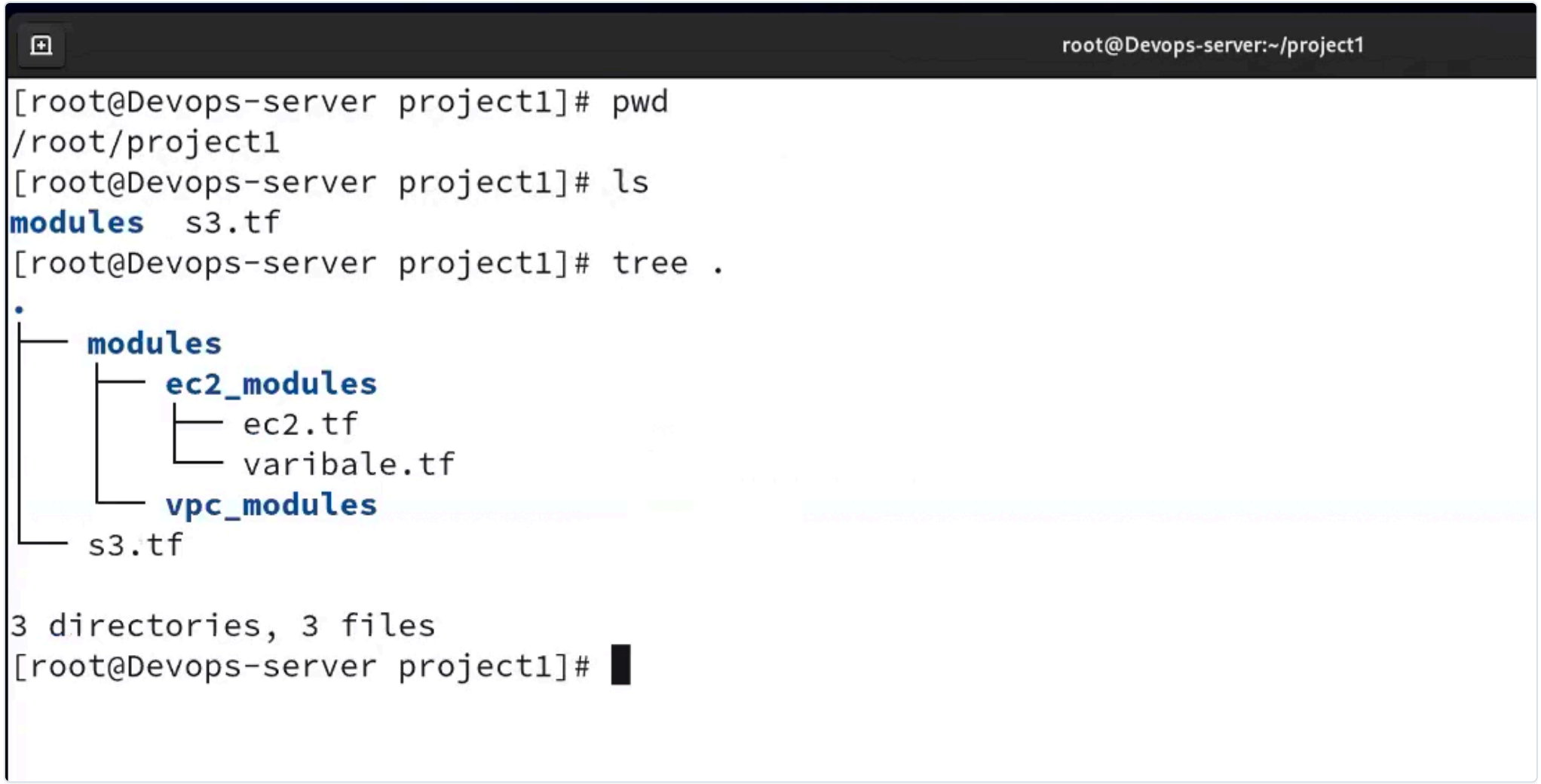
root@Devops-server:~/project1/modules/ec2_modules — vim varibale.tf

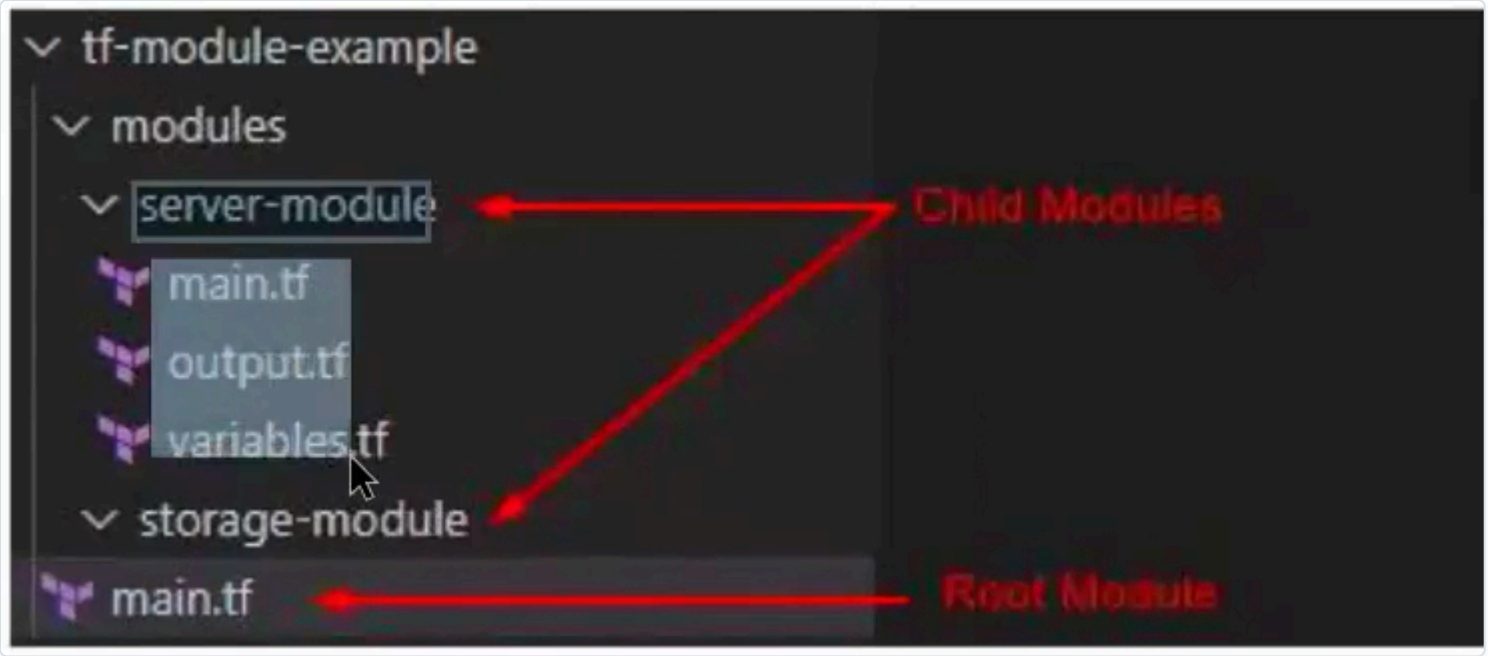
```
variable "instance_count" {
  type    = number
  default = 1
}
```

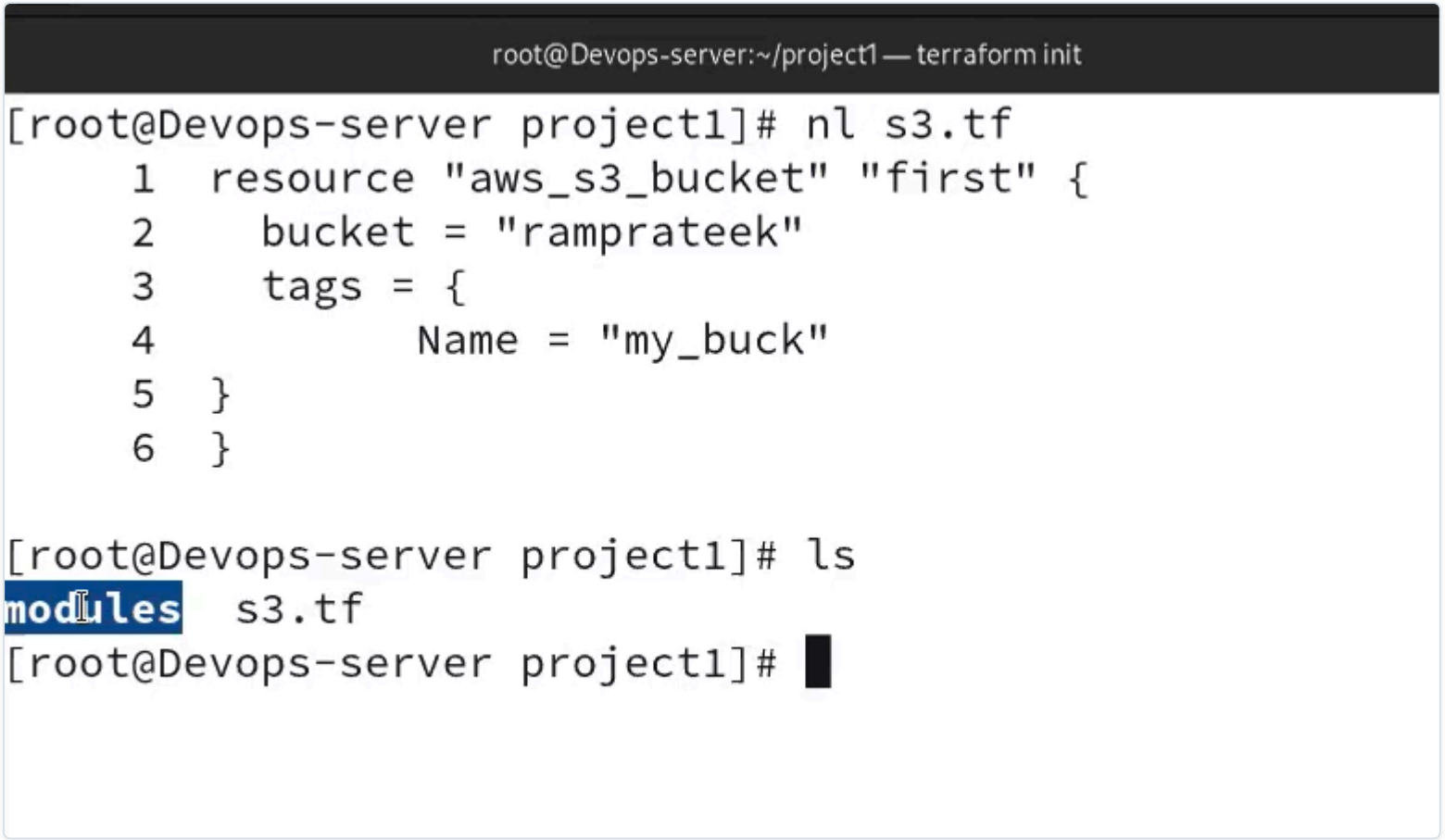
root@Devops-server:~/project1/modules/ec2_modules — vim ec2.tf

```
resource "aws_instance" "linux" {
  count         = var.instance_count
  ami           = "ami-0cca134ec43cf708f"
  instance_type = "t2.micro"
  key_name      = "ram_aws"

  tags = {
    Name = "web-server-${count.index}"
  }
}
```


root@Devops-server:~/project1
[root@Devops-server project1]# pwd
/root/project1
[root@Devops-server project1]# ls
modules s3.tf
[root@Devops-server project1]# tree .
.
├── modules
│   ├── ec2_modules
│   │   ├── ec2.tf
│   │   └── varibale.tf
│   └── vpc_modules
└── s3.tf
3 directories, 3 files
[root@Devops-server project1]#


tf-module-example
modules
server-module
main.tf
output.tf
variables.tf
storage-module
main.tf
Child Modules
Root Module


root@Devops-server:~/project1 — terraform init
[root@Devops-server project1]# nl s3.tf
1 resource "aws_s3_bucket" "first" {
2 bucket = "ramprateek"
3 tags = {
4 Name = "my_buck"
5 }
6 }
[root@Devops-server project1]# ls
modules s3.tf
[root@Devops-server project1]#

```
[root@Devops-server ec2_modules]# ll
total 8
-rw-r--r--. 1 root root 229 Jun 30 08:43 ec2.tf
-rw-r--r--. 1 root root  64 Jun 30 08:43 varibale.tf
[root@Devops-server ec2_modules]# cat ec2.tf
resource "aws_instance" "linux" {
  count         = var.instance_count
  ami           = "ami-0ca134ec43cf708f"
  instance_type = "t2.micro"
  key_name      = "ram_aws"

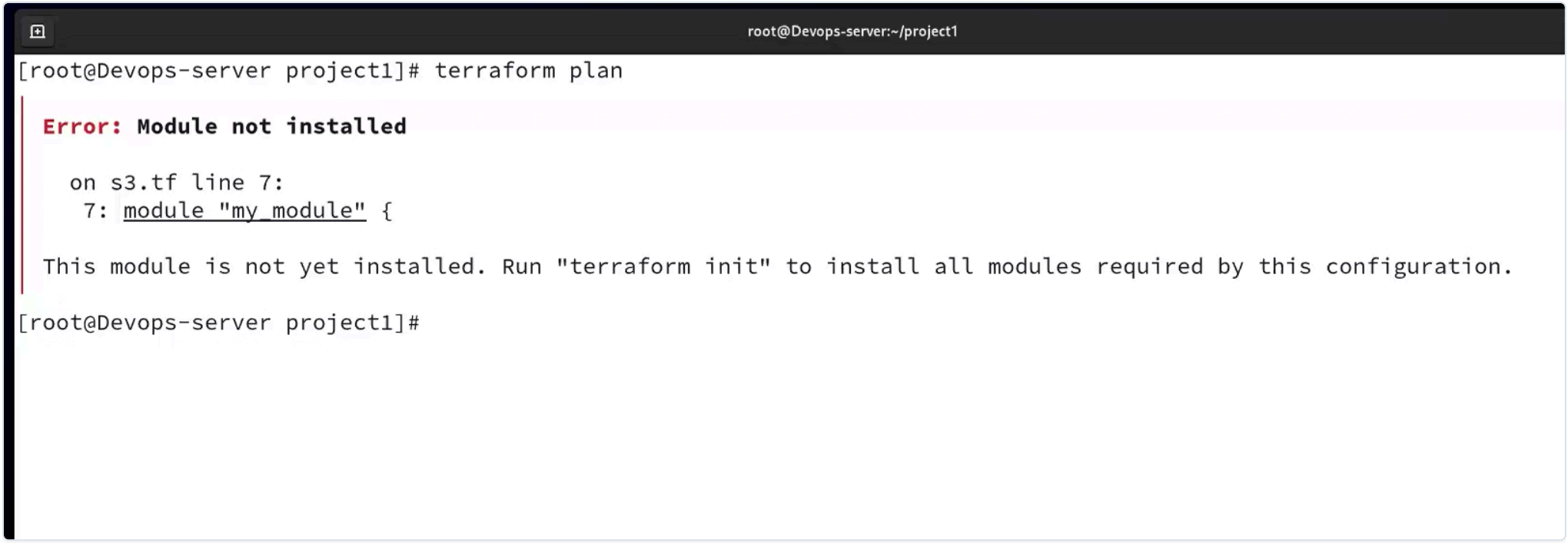
  tags = {
    Name = "web-server-${count.index}"
  }
}

[root@Devops-server ec2_modules]# cat varibale.tf
variable "instance_count" {
  type    = number
  default = 1
}

[root@Devops-server ec2_modules]#
```

root@Devops-server:~/project1 — vim s3.tf

```
resource "aws_s3_bucket" "first" {
  bucket = "ramprateek"
  tags = {
        Name = "my_buck"
}
}
module "my_module" {
        source = "./modules/ec2_modules"
        instance_count = 2 # Passing variable to control number of EC2 instances inside the module
}
```


root@Devops-server:~/project1
[root@Devops-server project1]# terraform plan
Error: Module not installed
on s3.tf line 7:
7: module "my_module" {
This module is not yet installed. Run "terraform init" to install all modules required by this configuration.
[root@Devops-server project1]#

```
[root@Devops-server project1]# terraform init
Initializing the backend...
Initializing modules...
- my_module in modules/ec2_modules
Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.52.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.
```

```
[root@Devops-server project1]# cd modules/
[root@Devops-server modules]# cd vpc_modules/
[root@Devops-server vpc_modules]# ls
[root@Devops-server vpc_modules]# vim
```

root@Devops-server:~/project1/modules/vpc_modules — vim vpc.tf

```
provider "aws" {
        region = "ap-south-1"
}
# Define variables Here
variable "vpc_cidr" {
        default = "10.1.0.0/16"
        description = "cidr for our custom vpc"
}
variable "subnet_cidr" {
        default = "10.1.1.0/24"
        description = "cidr for subnet"
}
variable "availability_zone" {
        default = "ap-south-1a"
        description = "AZ for subnet"
}
variable "instance_ami" {
        default = "ami-0cca134ec43cf708f"
        description = "default ami for instances"
}
variable "instance_type" {
        default = "t2.micro"
        description = "instance type for ec2"
}
```

```
variable "env_tag" {
        default = "production"
        description = "environment tag"
}
# code - creating vpc

resource "aws_vpc" "vpcone" {
        cidr_block = "${var.vpc_cidr}"
        tags = {
                Name = "${var.env_tag}"
        }
}

# code - creating IG and attaching it to VPC
resource "aws_internet_gateway" "vpcone-ig" {
        vpc_id = "${aws_vpc.vpcone.id}"
        tags = {
                Name = "${var.env_tag}"
        }
}
```

```
47 # code - create subnet inside our vpc
48 resource "aws_subnet" "subnet_public" {
49         vpc_id = "${aws_vpc.vpcone.id}"
50         cidr_block = "${var.subnet_cidr}"
51         map_public_ip_on_launch = "true"
52         availability_zone = "${var.availability_zone}"
53         tags = {
54                 Name = "${var.env_tag}"
55         }
56
57 }
58
59 # code - modifying route
60
61 resource "aws_route_table" "rtb_public" {
62         vpc_id = "${aws_vpc.vpcone.id}"
63         route {
64                 cidr_block = "0.0.0.0/0"
65                 gateway_id = "${aws_internet_gateway.vpcone-ig.id}"
66         }
67         tags = {
68                 Name = "${var.env_tag}"
69         }
70 }
71
```

```
# code - attaching subnets to route table
resource "aws_route_table_association" "rta_subnet_public" {
        subnet_id = "${aws_subnet.subnet_public.id}"
        route_table_id = "${aws_route_table.rtb_public.id}"
}

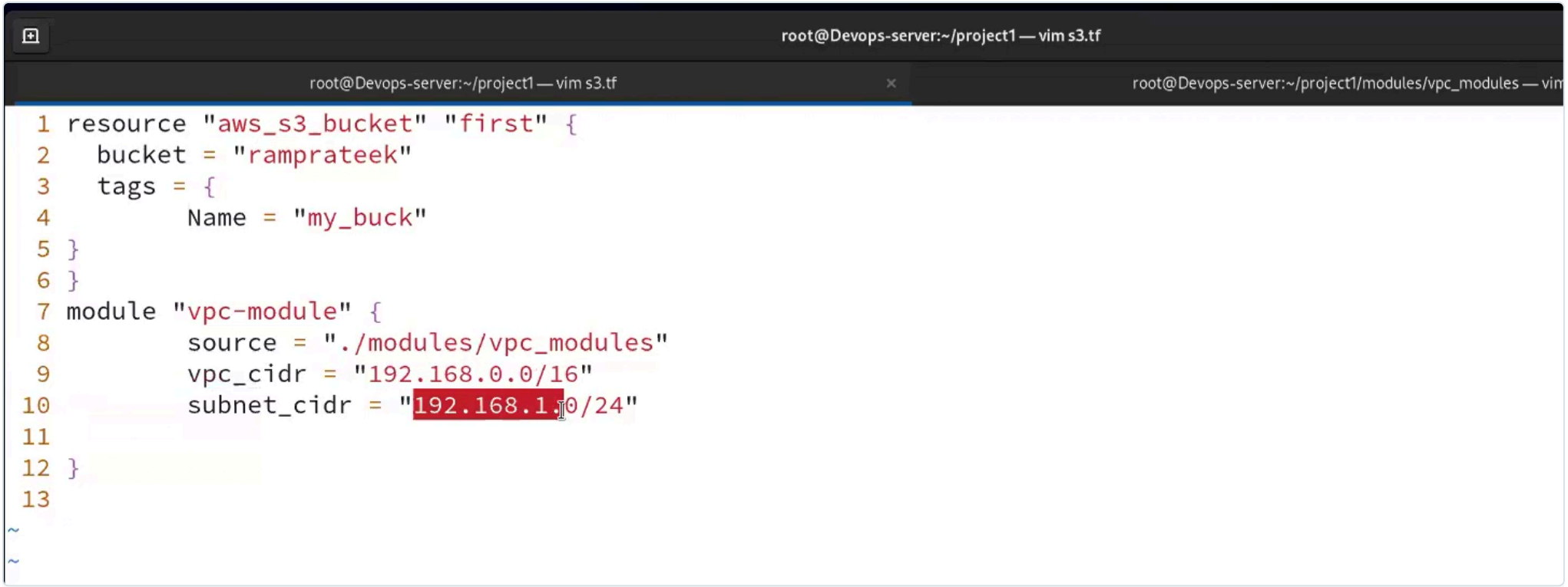
# code - create security group
resource "aws_security_group" "sg_newvpc" {
        name = "newvpc"
        vpc_id = "${aws_vpc.vpcone.id}"
        ingress {
                from_port = 22
                to_port = 22
                protocol = "tcp"
                cidr_blocks = ["0.0.0.0/0"]
        }

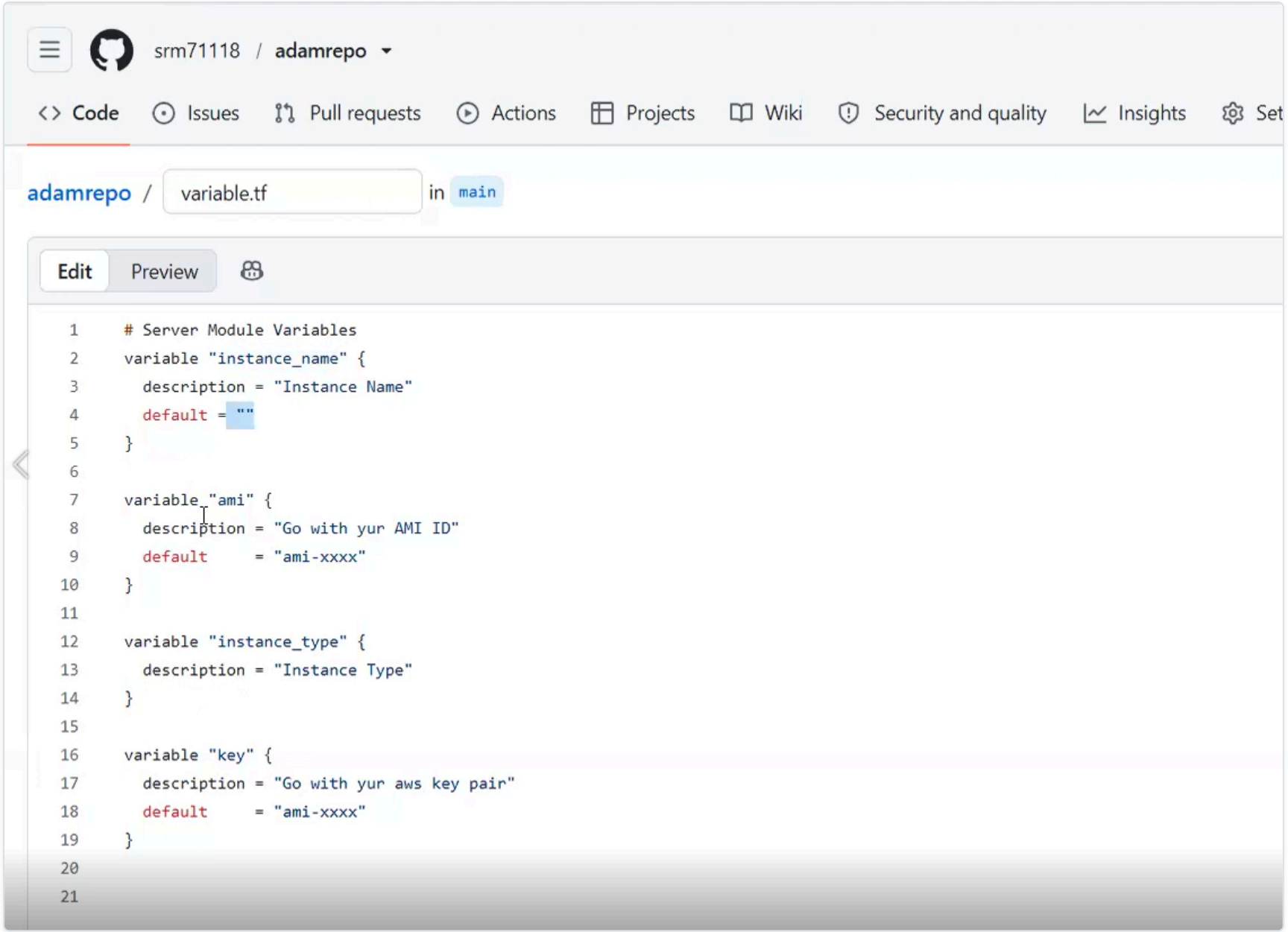
        egress {
                from_port = 0
```

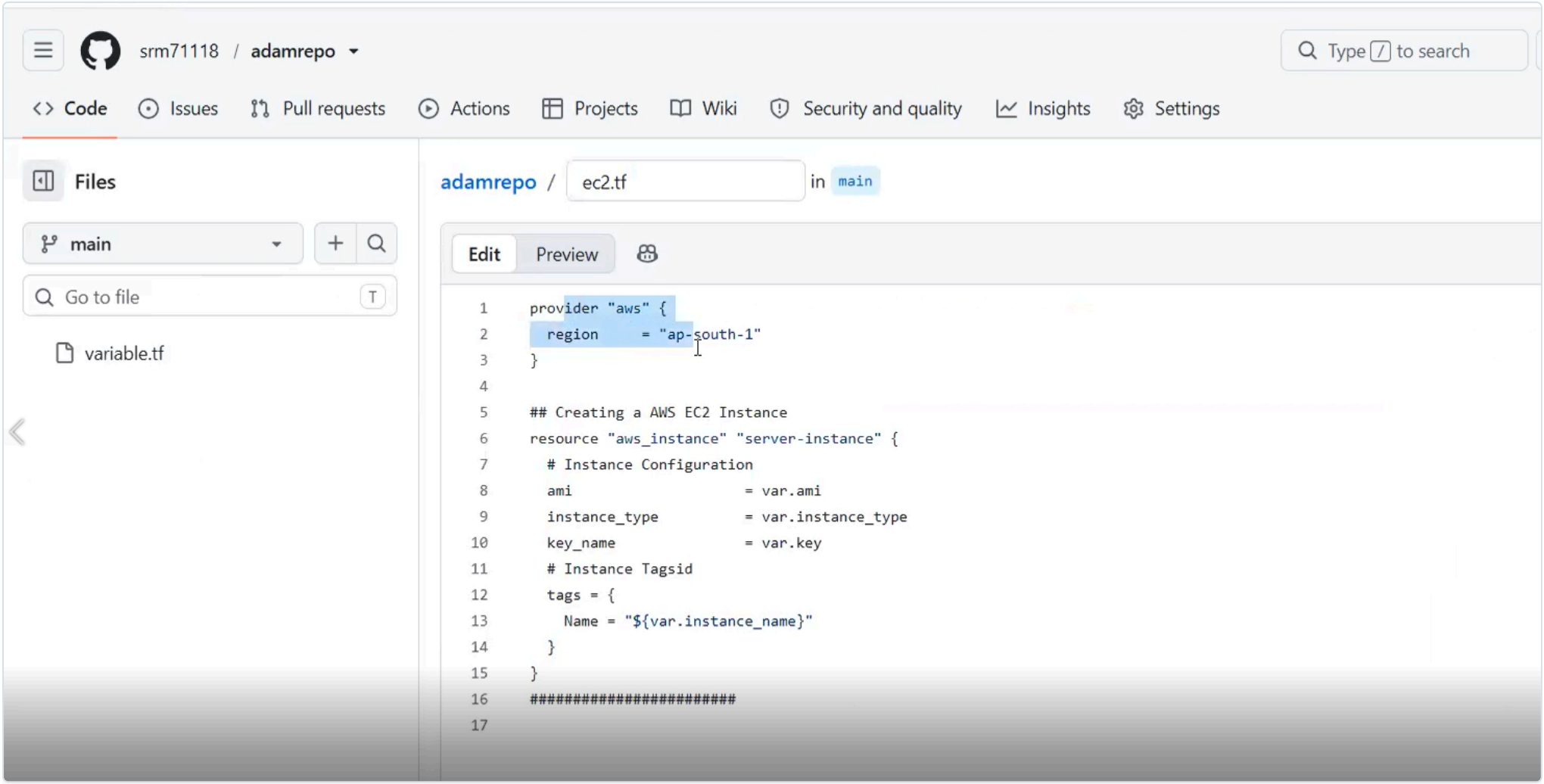
```
 87         }
 88
 89         egress {
 90                 from_port = 0
 91                 to_port = 0
 92                 protocol = "-1"
 93                 cidr_blocks = ["0.0.0.0/0"]
 94         }
 95
 96         tags = {
 97                 Name = "${var.env_tag}"
 98         }
 99
100 }
101 # code - create instance
102 resource "aws_instance" "test" {
103         ami = "${var.instance_ami}"
104         instance_type = "${var.instance_type}"
105         subnet_id = "${aws_subnet.subnet_public.id}"
106         vpc_security_group_ids = ["${aws_security_group.sg_newvpc.id}"]
107         tags = {
108                 Name = "${var.env_tag}"
109         }
110 }
111
                                                                    111,0-1
```

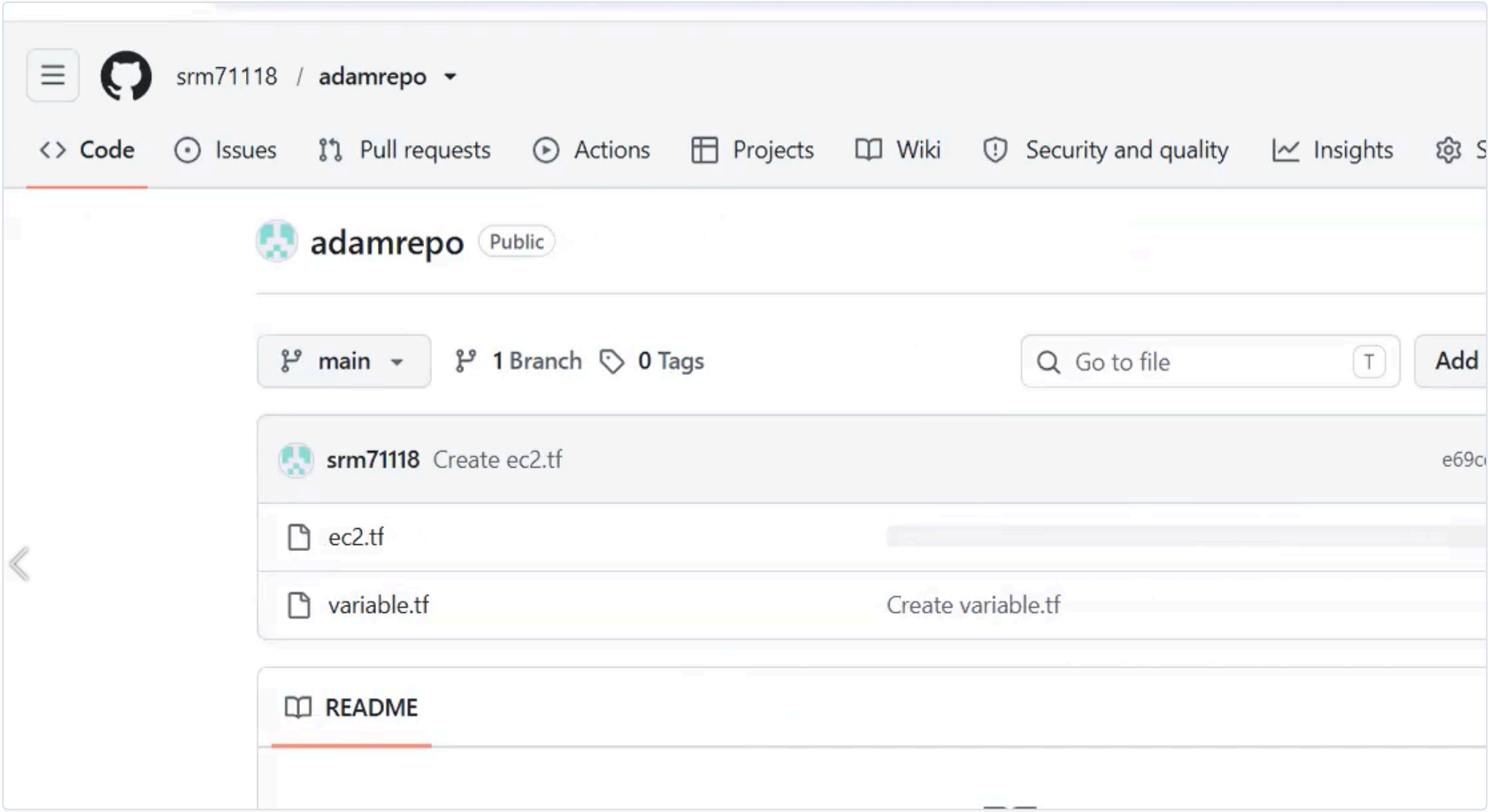
root@Devops-server:~/project1 — vim s3.tf

```
resource "aws_s3_bucket" "first" {
  bucket = "ramprateek"
  tags = {
        Name = "my_buck"
}
}
module "vpc-module" {
        source = "./modules/vpc_modules"
}
```


root@Devops-server:~/project1 — vim s3.tf
root@Devops-server:~/project1 — vim s3.tf
root@Devops-server:~/project1/modules/vpc_modules — vim
1 resource "aws_s3_bucket" "first" {
2 bucket = "ramprateek"
3 tags = {
4 Name = "my_buck"
5 }
6 }
7 module "vpc-module" {
8 source = "./modules/vpc_modules"
9 vpc_cidr = "192.168.0.0/16"
10 subnet_cidr = "192.168.1.0/24"
11
12 }
13


srm71118 / adamrepo
Code
Issues
Pull requests
Actions
Projects
Wiki
Security and quality
Insights
Set
adamrepo / variable.tf in main
Edit
Preview
Server Module Variables
variable "instance_name" {
description = "Instance Name"
default = ""
}
variable "ami" {
description = "Go with yur AMI ID"
default = "ami-xxxx"
}
variable "instance_type" {
description = "Instance Type"
}
variable "key" {
description = "Go with yur aws key pair"
default = "ami-xxxx"
}


srm71118 / adamrepo
Type / to search
Code
Issues
Pull requests
Actions
Projects
Wiki
Security and quality
Insights
Settings
Files
main
Go to file T
variable.tf
adamrepo / ec2.tf in main
Edit
Preview
provider "aws" {
region = "ap-south-1"
}
Creating a AWS EC2 Instance
resource "aws_instance" "server-instance" {
Instance Configuration
ami = var.ami
instance_type = var.instance_type
key_name = var.key
Instance Tagsid
tags = {
Name = "${var.instance_name}"
}
}
########################

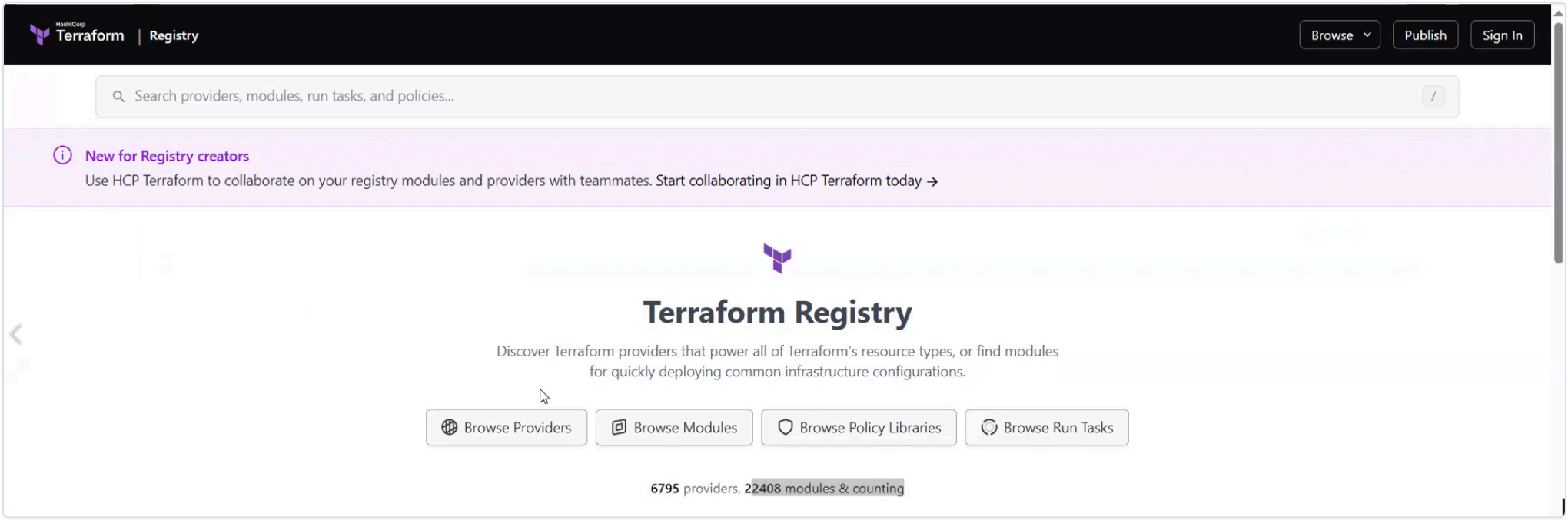

srm71118 / adamrepo
Code
Issues
Pull requests
Actions
Projects
Wiki
Security and quality
Insights
adamrepo
Public
main
1 Branch
0 Tags
Go to file
T
Add
srm71118 Create ec2.tf
ec2.tf
variable.tf
Create variable.tf
README

Root calls `module "adam-module"` with a GitHub source + 4 inputs (note the stray } on line 15)

root@Devops-server:~/project1 — vim s3.tf

```
 1 resource "aws_s3_bucket" "first" {
 2   bucket = "ramprateek"
 3   tags = {
 4         Name = "my_buck"
 5 }
 6 }
 7 module "adam-module" {
 8 source =  "github.com/srm71118/d"
 9     ami = "ami-0cca134ec43cf708f"
10     instance_type = "t2.micro"
11     instance_name = "Demo-server"
12     key = "ram_aws"
13 }
14
15 }
16
~
~
```

```
Plan: 2 to add, 0 to change, 0 to destroy.
aws_s3_bucket.first: Creating...
module.adam-module.aws_instance.server-instance: Creating...
aws_s3_bucket.first: Creation complete after 2s [id=ramprateek]
```


HashiCorp
Terraform | Registry
Browse
Publish
Sign In
Search providers, modules, run tasks, and policies...
/
New for Registry creators
Use HCP Terraform to collaborate on your registry modules and providers with teammates. Start collaborating in HCP Terraform today →
Terraform Registry
Discover Terraform providers that power all of Terraform's resource types, or find modules for quickly deploying common infrastructure configurations.
Browse Providers
Browse Modules
Browse Policy Libraries
Browse Run Tasks
6795 providers, 22408 modules & counting

root@Devops-server:~/project1 — vim main.tf

```
module "s3_bucket" {
  source = "terraform-aws-modules/s3-bucket/aws"

  bucket = "lalalalal123212344"
  acl    = "private"

  control_object_ownership = true
  object_ownership         = "ObjectWriter"

  versioning = {
    enabled = true
  }
}
~
~
```

```
[root@Devops-server project1]# terraform init
Initializing the backend...
Initializing modules...
Downloading registry.terraform.io/terraform-aws-modules/s3-bucket/aws 5.14.1 for s3_bucket...
- s3_bucket in .terraform/modules/s3_bucket
Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.52.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
[root@Devops-server project1]#
```

```
[root@Devops-server project1]# nl main.tf
     1  module "s3_bucket" {
     2    source = "terraform-aws-modules/s3-bucket/aws"

     3    bucket = "lalalalal123212344"
     4    acl    = "private"

     5    control_object_ownership = true
     6    object_ownership         = "ObjectWriter"

     7    versioning = {
     8      enabled = true
     9    }
    10  }
[root@Devops-server project1]# ls .terraform/modules/
adam-module  modules.json  s3_bucket
[root@Devops-server project1]#
```

Destroy (2 destroyed), `rm -rf *`, init re-downloads 5.14.1


```
Destroy complete! Resources: 2 destroyed.
[root@Devops-server project1]# rm -rf *
[root@Devops-server project1]# vim main.tf
[root@Devops-server project1]# terraform init
[root@Devops-server project1]# terraform init
Initializing the backend...
Initializing modules...
Downloading registry.terraform.io/terraform-aws-modules/s3-bucket/aws 5.14.1 for s3_bucket...
- s3_bucket in .terraform/modules/s3_bucket
Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
```